Analysis for the usage of partial classes and abstract classes & Model Preperation in BusBooking System

BusBookingSystem source code GitHub Url:
https://github.com/Padmavathi-SJ/Bus_Ticket_Booking

Abstract Class:

An abstract class is a bluebrint that cannot be instantiated(inherting)(we cannot create objects from this class).

Ex: Vehicle class is a abstract class here,

- I cannot create a vehicle object from this class, (Vehicle myVehicle = new Vehicle();)
- But I cannot create a Car or Bike objects from this class.(Car myCar = new Car();)

- abstract classes cannot be instantiated,
- it can have constructors
- it can have fields to store data
- it can have implemented methods
- it can have abstract methods (no body methods must be overridden)
- it can have virtual methods(can override but not required)
  can have properties(with getter, setter)

→ when multiple classes share common code → I need to use abstract classes.

→ here I have different logic for 3 different users (admin, bus operator, customer) the logics are different for all 3, so I cannot use abstract class with common methods or common properties.

→ if I need to provide ‘default implementation’ I need to use abstract classes.

→ so ‘bus adding’ with necessary bus details feature same for admin and bus operator , so I can use abstract class with default bus adding properties,

similarity	**Admin command**	**Operator command**	**Same?**
properties	All bus details	All bus details	Same
validation	Same attributes	Same attributes	Same
Bus creation logic	Creates bus entity	Creates bus entity	same
Only differences are	Status =approved by default for admins	Status = pending(needs admin approval)	different
	Isavailabl=true	isAvailable=false	Different
	operatoriId optional	operatorId required	different

→ Now I have created a **‘BaseBusCommand.cs’** abstract class:

/BusBooking.Application/Admin/Commands/BaseBusCommand.cs:

```
using MediatR;
using System.ComponentModel.DataAnnotations;


namespace BusBooking.Application.Commands
{
   public abstract class BaseBusCommand : IRequest<Guid>
   {
      // Common properties - all bus commands need these
      [Required] public Guid RouteId { get; set; }
      [Required] public string BusNumber { get; set; } = string.Empty;
      [Required] public string BusName { get; set; } = string.Empty;
      [Required] public string BusType { get; set; } = string.Empty;
      [Required] public int TotalSeats { get; set; }
      [Required] public int FemaleSeats { get; set; }
```

```
[Required] public int MaleSeats { get; set; }
[Required] public double BasePrice { get; set; }
public string Amenities { get; set; } = string.Empty;

// Template method - each derived class provides its own logic
protected abstract BusStatus GetInitialStatus();
protected abstract bool GetInitialAvailability();
protected abstract Guid? GetOperatorId();

// Factory method - creates Bus from common properties
public Bus ToBusEntity()
{
    return new Bus
    {
        Id = Guid.NewGuid(),
        RouteId = this.RouteId,
        BusNumber = this.BusNumber,
        BusName = this.BusName,
        BusType = this.BusType,
        TotalSeats = this.TotalSeats,
        FemaleSeats = this.FemaleSeats,
        MaleSeats = this.MaleSeats,
        BasePrice = this.BasePrice,
        Amenities = this.Amenities,
```

```
            OperatorId = GetOperatorId(),
            Status = GetInitialStatus(),
            IsAvailable = GetInitialAvailability(),
            CreatedAt = DateTime.UtcNow
        };
    }
  }
}
```

→ **AdminAdddBusCommand** and Operator's **CreateBusCommand** , both were having same the properties to add bus(so many duplicate lines of code)

→ now single command '**BaseBusCommand'** has the properties of bus to ceate new buses.

→**adminAddBusCommand** and Operator busadding Command can get inherited from **BaseBusCommand**:

BaseBusCommand - Single source

```
public abstract class BaseBusCommand : IRequest<Guid>
{
   [Required] public Guid RouteId { get; set; }
   [Required] public string BusNumber { get; set; } = string.Empty;
   [Required] public string BusName { get; set; } = string.Empty;
   [Required] public string BusType { get; set; } = string.Empty;
   [Required] public int TotalSeats { get; set; }
   [Required] public int FemaleSeats { get; set; }
```

```
    [Required] public int MaleSeats { get; set; }
    [Required] public double BasePrice { get; set; }
    public string Amenities { get; set; } = string.Empty;

    // Abstract methods - each command defines its own behavior
    protected abstract BusStatus GetInitialStatus();
    protected abstract bool GetInitialAvailability();
    protected abstract Guid? GetOperatorId();
}

//  Admin command - NOW only 15 lines!
public class AdminAddBusCommand : BaseBusCommand
{
    public Guid? OperatorId { get; set; }

    protected override BusStatus GetInitialStatus() => BusStatus.Approved;
    protected override bool GetInitialAvailability() => true;
    protected override Guid? GetOperatorId() => OperatorId;
}

// Operator command - NOW only 12 lines!
public class CreateBusCommand : BaseBusCommand
{
    public Guid OperatorId { get; set; }
```

```
    protected override BusStatus GetInitialStatus() => BusStatus.Pending;
    protected override bool GetInitialAvailability() => false;
    protected override Guid? GetOperatorId() => OperatorId;
}
```

→ If I need to change or add any new properties, I need to change in **BasebusCommand** only!

→ It is giving consistency across all bus commands!

→ and **BaseBusCommandHandler** →common bus creation logic

→ after changing with Base handlers:

Admin Add Bus Handler - Before vs After

// BEFORE: 30 lines of repetitive code

```
public class AdminAddBusCommandHandler :
IRequestHandler<AdminAddBusCommand, Guid>
{
    private readonly IAppDbContext _context;

    public AdminAddBusCommandHandler(IAppDbContext context)
    {
        _context = context;
    }
```

```
    public async Task<Guid> Handle(AdminAddBusCommand request,
CancellationToken ct)
    {
        var bus = new Bus
        {
            Id = Guid.NewGuid(),
            RouteId = request.RouteId,
            BusNumber = request.BusNumber,
            BusName = request.BusName,
            BusType = request.BusType,
            TotalSeats = request.TotalSeats,
            FemaleSeats = request.FemaleSeats,
            MaleSeats = request.MaleSeats,
            BasePrice = request.BasePrice,
            Amenities = request.Amenities,
            OperatorId = request.OperatorId,
            Status = BusStatus.Approved,
            IsAvailable = true,
            CreatedAt = DateTime.UtcNow
        };

        _context.Buses.Add(bus);
        await _context.SaveChangesAsync(ct);
        return bus.Id;
    }
```

```
}

// AFTER: 5 lines (90% reduction!)
public class AdminAddBusCommandHandler :
BaseBusCommandHandler<AdminAddBusCommand>
{
    public AdminAddBusCommandHandler(IAppDbContext context) :
base(context) { }

    // No need to override anything! Base handler does all work.
    // Admin has no special validation, so nothing to add!
}
```

Operator Add Bus Handler - Before vs After

```
// BEFORE: 50 lines of complex code
public class CreateBusCommandHandler : IRequestHandler<CreateBusCommand,
Guid>
{
    private readonly IAppDbContext _context;

    public CreateBusCommandHandler(IAppDbContext context)
    {
        _context = context;
    }
```

```
    public async Task<Guid> Handle(CreateBusCommand request,
CancellationToken ct)
    {
        // 1. Complex operator validation (20 lines)
        var busOperator = await _context.BusOperators
            .Include(o => o.User)
            .FirstOrDefaultAsync(o => o.UserId == request.OperatorId, ct);


        if (busOperator == null)
        {
            var user = await _context.Users.FindAsync(new object[] {
request.OperatorId }, ct);
            if (user != null && user.Role == UserRole.BusOperator)
            {
                busOperator = new BusOperator
                {
                    Id = Guid.NewGuid(),
                    UserId = user.Id,
                    CompanyName = user.FullName + " Transport",
                    LicenseNumber = "PENDING",
                    Address = "Default Address",
                    Status = OperatorStatus.Approved,
                    CreatedAt = DateTime.UtcNow
                };
```

```
        _context.BusOperators.Add(busOperator);
        await _context.SaveChangesAsync(ct);
    }
}

if (busOperator == null)
    throw new Exception($"Unauthorized");

// 2. Create bus (20 lines of property assignment)
var bus = new Bus
{
    Id = Guid.NewGuid(),
    OperatorId = busOperator.Id,
    RouteId = request.RouteId,
    BusNumber = request.BusNumber,
    BusName = request.BusName,
    BusType = request.BusType,
    TotalSeats = request.TotalSeats,
    FemaleSeats = request.FemaleSeats,
    MaleSeats = request.MaleSeats,
    BasePrice = request.BasePrice,
    Amenities = request.Amenities,
    Status = BusStatus.Pending,
    IsAvailable = false,
```

```
            CreatedAt = DateTime.UtcNow
        };

        // 3. Save (5 lines)
        _context.Buses.Add(bus);
        await _context.SaveChangesAsync(ct);
        return bus.Id;
    }
}

// AFTER: 25 lines (50% reduction, and much cleaner!)
public class CreateBusCommandHandler :
BaseBusCommandHandler<CreateBusCommand>
{
    public CreateBusCommandHandler(IAppDbContext context) : base(context) { }

    // ONLY override the operator validation part (15 lines)
    protected override async Task BeforeCreateAsync(CreateBusCommand request,
CancellationToken ct)
    {
        var busOperator = await _context.BusOperators
            .Include(o => o.User)
            .FirstOrDefaultAsync(o => o.UserId == request.OperatorId, ct);

        if (busOperator == null)
```

```
        {
            var user = await _context.Users.FindAsync(new object[] {
request.OperatorId }, ct);
            if (user != null && user.Role == UserRole.BusOperator)
            {
                busOperator = new BusOperator
                {
                    Id = Guid.NewGuid(),
                    UserId = user.Id,
                    CompanyName = user.FullName + " Transport",
                    LicenseNumber = "PENDING",
                    Address = "Default Address",
                    Status = OperatorStatus.Approved,
                    CreatedAt = DateTime.UtcNow
                };
                _context.BusOperators.Add(busOperator);
                await _context.SaveChangesAsync(ct);
            }
        }

        if (busOperator == null)
            throw new Exception($"Unauthorized: No operator profile linked to User
ID {request.OperatorId}");

        // Store operator ID for bus creation
```

```
        request.OperatorId = busOperator.Id;
    }
}
```

Partial classes:

A partial class allows us to split a single class definition across multiple files. So at compile time, C# combines all parts into one complete class.

→when to use partial classes:

→ when the entities have huge properties like around 30, more than 30 properties, I could use partial classes,

but my entities have just pure properties alone:

/BusBooking.Domain/Entities/Bus.cs:

```
using BusBooking.Domain.Common;
using BusBooking.Domain.Enums;

namespace BusBooking.Domain.Entities;

public class Bus : BaseEntity
{
    public Guid? OperatorId { get; set; } // Nullable if added by Admin directly
    public Guid RouteId { get; set; }
    public string BusNumber { get; set; } = string.Empty;
    public string BusName { get; set; } = string.Empty;
    public string BusType { get; set; } = string.Empty; // e.g. "AC Sleeper", "Non-AC Seater"
```

```
    public int TotalSeats { get; set; }
    public double BasePrice { get; set; }
    public string Amenities { get; set; } = string.Empty; // e.g. "Wifi, Water, Charging Point"
    public string? Description { get; set; }
    public BusStatus Status { get; set; } = BusStatus.Pending;
    public bool IsAvailable { get; set; } = true;
    public int FemaleSeats { get; set; }
    public int MaleSeats { get; set; }

    // Approval tracking
    public Guid? ApprovedBy { get; set; }
    public DateTime? ApprovedAt { get; set; }
    public string? RejectionReason { get; set; }

    // Navigation
    public BusOperator? Operator { get; set; }
    public BusRoute Route { get; set; } = null!;
    public ICollection<SeatLayout> SeatLayouts { get; set; } = new List<SeatLayout>();
    public ICollection<Trip> Trips { get; set; } = new List<Trip>();
}
```

/BusBooking.Domain/Entities/BusRoute.cs:

```
using BusBooking.Domain.Common;
```

```
namespace BusBooking.Domain.Entities;

public class BusRoute : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Source { get; set; } = string.Empty;
    public string Destination { get; set; } = string.Empty;
    public double DistanceKm { get; set; }
    public bool IsActive { get; set; } = true;

    // Navigation
    public ICollection<RouteStop> Stops { get; set; } = new List<RouteStop>();
    public ICollection<Trip> Trips { get; set; } = new List<Trip>();
}
```

BusBooking.Domain/Entities/Booking.cs

```
using BusBooking.Domain.Common;
using BusBooking.Domain.Enums;

namespace BusBooking.Domain.Entities;

public class Booking : BaseEntity
{
```

```
    public Guid CustomerId { get; set; }
    public Guid TripId { get; set; }
    public DateTime BookingDate { get; set; } = DateTime.UtcNow;
    public decimal TotalAmount { get; set; }
    public BookingStatus Status { get; set; } = BookingStatus.Pending;
    public string? CancellationReason { get; set; }

    // Navigation
    public User Customer { get; set; } = null!;
    public Trip Trip { get; set; } = null!;
    public ICollection<BookingSeat> Seats { get; set; } = new
List<BookingSeat>();
    public Payment? Payment { get; set; }
}
```

→ I separated **validation** methods in **commands**(BusBooking.Application/…)

→ **displaying logics** are in **DTOs**(BusBooking.Application/…)

→ if large classes with mixed responsibilities, I could use partial classes to split them into organization structure, so it will be easy to reuse, update and make changes!

→ and also if more than 1 developer is working in a project, then using partial classes is good practice, because the structure is very cleaner and maintainable and easy to find problems and update those too!

→ in my case I am only the developer working in busbooking application , and also so far I have very less things to handle and maintain , in future if I implement this project with more cases, then I should definitly go for partial classes! Although I am only person working in this, it could be easy for maintaining things in backend!

→I could do validation, verification, storage, anything in a separate/partial class, and I should properly make connection with all, so that in compilation time, all things could combine to make a single class!

Model Preperation:

1.I have Base Entity:

// BusBooking.Domain/Common/BaseEntity

```
public abstract class BaseEntity
{
   public Guid Id { get; set; }
   public DateTime CreatedAt { get; set; }
   public DateTime? UpdatedAt { get; set; }
   public bool IsDeleted { get; set; }
   // Common properties for ALL entities
}
```

2.I have clean Entity Models:

/BusBooking.Domain/Entities/Bus.cs:

```
using BusBooking.Domain.Common;

using BusBooking.Domain.Enums;
```

```
namespace BusBooking.Domain.Entities;


public class Bus : BaseEntity
{
    public Guid? OperatorId { get; set; } // Nullable if added by Admin directly
    public Guid RouteId { get; set; }
    public string BusNumber { get; set; } = string.Empty;
    public string BusName { get; set; } = string.Empty;
    public string BusType { get; set; } = string.Empty; // e.g. "AC Sleeper", "Non-AC Seater"
    public int TotalSeats { get; set; }
    public double BasePrice { get; set; }
    public string Amenities { get; set; } = string.Empty; // e.g. "Wifi, Water, Charging Point"
    public string? Description { get; set; }
    public BusStatus Status { get; set; } = BusStatus.Pending;
    public bool IsAvailable { get; set; } = true;
    public int FemaleSeats { get; set; }
    public int MaleSeats { get; set; }


    // Approval tracking
    public Guid? ApprovedBy { get; set; }
    public DateTime? ApprovedAt { get; set; }
    public string? RejectionReason { get; set; }
```

```
    // Navigation
    public BusOperator? Operator { get; set; }
    public BusRoute Route { get; set; } = null!;
    public ICollection<SeatLayout> SeatLayouts { get; set; } = new
List<SeatLayout>();
    public ICollection<Trip> Trips { get; set; } = new List<Trip>();
}
```

3.I have enums also:

/BusBooking.Domain/Enums/

BusStatus.cs:

```
namespace BusBooking.Domain.Enums;

public enum BusStatus
{
    Pending = 1,
    Approved = 2,
    Disabled = 3,
    Rejected = 4
}
```

SeatType.cs:

```
namespace BusBooking.Domain.Enums;

public enum SeatType
```

```
{
    Window = 1,
    Aisle = 2,
    Sleeper = 3,
    Seater = 4
}
```

BookingStatus.cs:

```
namespace BusBooking.Domain.Enums;


public enum BookingStatus
{
    Pending = 1,
    Confirmed = 2,
    Cancelled = 3,
    Expired = 4
}
```

→ I have the navigation properties also.

→ so here I can get the point of model preparation:

→ without model preparation, everything is scattered, I could make mistakes, and its again hard to maintain, no data validation, no relations between data,

→ so with model preparation ,

→ I can prepare the model(blueprint of design) and I can use it everywhere .

→ so data will be organized in one place, and I can have validation also , if defining in one place, then it works everywhere perfectly. And making strong relations.